

Cálculo Paralelo do Conjunto de Mandelbrot

Versão Híbrida MPI + OpenMP — Trabalho 4

Bernardo Luiz Padilha Zamin João Pedro Sostruznik Sotero da Cunha

Programação Paralela — PUCRS, Escola Politécnica

2026

O problema

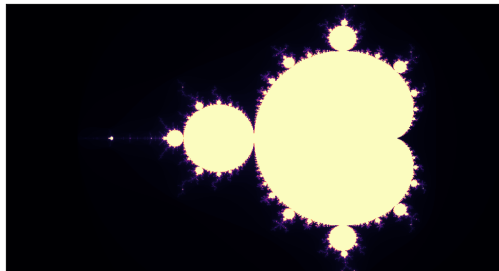
Conjunto de Mandelbrot — imagem **7680×4320** (33 MP).

Para cada pixel c : itera-se $z \leftarrow z^2 + c$ até $|z| > 2$ ou atingir `max_iter`.

- Pontos **internos** ao conjunto $\Rightarrow$ custam `max_iter` iterações.
- Pontos **externos** $\Rightarrow$ escapam cedo.

$\Rightarrow$ **carga altamente irregular** entre pixels/linhas.

$\Rightarrow$ exige **balanceamento dinâmico** (não dá para dividir igualmente).



Cor = nº de iterações até escapar.

Objetivo: dois níveis de paralelismo

Versão híbrida MPI + OpenMP, escalando em **4 nós** do cluster Atlantica.

Nível 1 — entre nós (MPI)

Memória **distribuída**. Distribui blocos de linhas da imagem entre os nós.

Um processo pesado MPI por nó.

Nível 2 — dentro do nó (OpenMP)

Memória **compartilhada**. Várias threads calculam as linhas do bloco.

Workpool sem mestre.

Coordenador (rank 0) → blocos de linhas sob demanda → 4 trabalhadores (16 threads cada)

- **Coordenador** (rank 0): guarda a imagem e distribui **blocos de linhas sob demanda**.
- **Trabalhador**: toma a iniciativa
 - ① pede trabalho (TAG_REQUEST);
 - ② recebe um bloco (TAG_TASK) ou o fim (TAG_STOP);
 - ③ calcula e devolve o resultado (TAG_RESULT).
- **Balanceamento natural**: quem termina antes pede mais $\Rightarrow$ ninguém fica ocioso, mesmo com a carga irregular.
- Reaproveita *exatamente* o modelo Coordenador/Trabalhador da versão MPI pura.

Nível 2 — OpenMP: workpool *sem mestre*

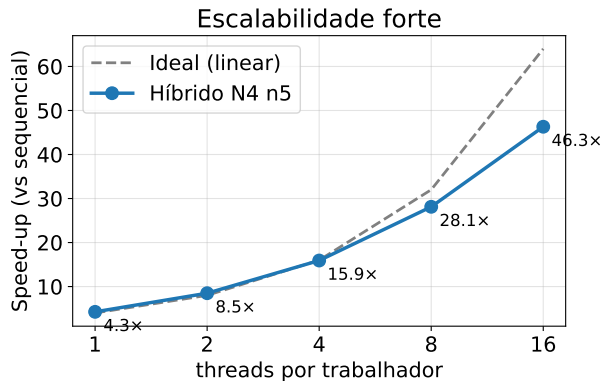
Todas as threads trabalham; o controle está numa **estrutura de dados** compartilhada (não há thread mestre).

```
typedef struct { int next; int total; } WorkPool;    // estrutura compartilhada

WorkPool pool = { .next = 0, .total = task->num_rows };
#pragma omp parallel                                // TODAS as threads entram
{
    while (1) {
        int row;
        #pragma omp atomic capture                    // retira a proxima linha
        { row = pool.next; pool.next++; }           // ... atomicamente
        if (row >= pool.total) break;                // pool vazio -> termina
        /* ... calcula os 7680 pixels da linha 'row' ... */
    }
}
```

Granularidade de **1 linha**; OMP_NUM_THREADS=16 (2 threads por núcleo físico, HT).

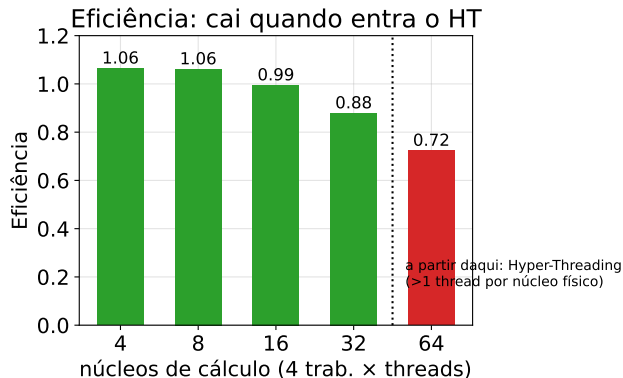
- 4 nós, Intel Xeon E5520, **16 CPUs lógicas/nó** (8 núcleos $\times$ 2 HT).
- Híbrido roda **sempre em** `-N 4 -n 5`: 1 coordenador + 4 trabalhadores (coordenador *compartilha* o nó 0).
- Escala-se pelo **nº de threads** por trabalhador: $1 \rightarrow 16$ ($\Rightarrow 4 \rightarrow 64$ núcleos de cálculo).
- **Forte**: `max_iter=2000` fixo. **Fraca**: `max_iter = 1500  $\times$  threads`.
- Tempo com `MPI_Wtime` (antes do I/O). Baseline $T(1) = 93,19$ s.
- Speed-up $S = T(1)/T(p)$, Eficiência $E = S/p$, $p = 4 \times \text{threads}$.
- **Correção**: saída **byte-idêntica** à da versão sequencial (mesmo md5).



Speed-up de $4,3\times$ a $46\times$ ($4 \rightarrow 64$ núcleos).

- Até 16 núcleos: **quase ideal** — o workpool mantém todas as threads ocupadas.
- *Superlinear* no início ($E > 1$): subvetor menor por núcleo $\Rightarrow$ melhor uso de **cache**.
- Ganhos diminuem além dos núcleos físicos (próximo slide).

Por que a eficiência cai? Hyper-Threading

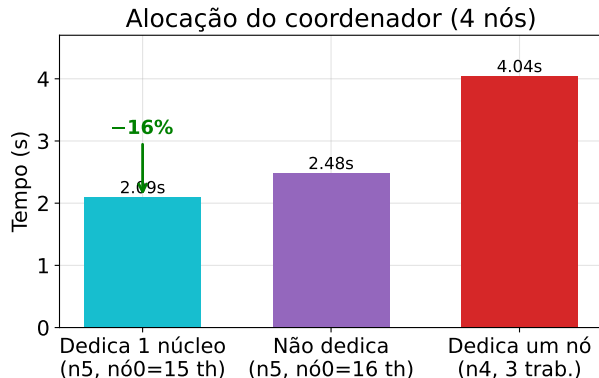


Eficiência: $1,06 \rightarrow 0,99 \rightarrow 0,88 \rightarrow \mathbf{0,72}$.

- **32 núcleos** = 8 threads/nó = *todos os núcleos físicos*: contenção de banda de memória.
- **64 núcleos** = 16 threads/nó = **Hyper-Threading** (2 threads/núcleo) — HT *não dobra* o desempenho.

⇒ o platô é **esperado**: é o limite físico da máquina, não do algoritmo.

Alocação do coordenador: dedicar um núcleo?

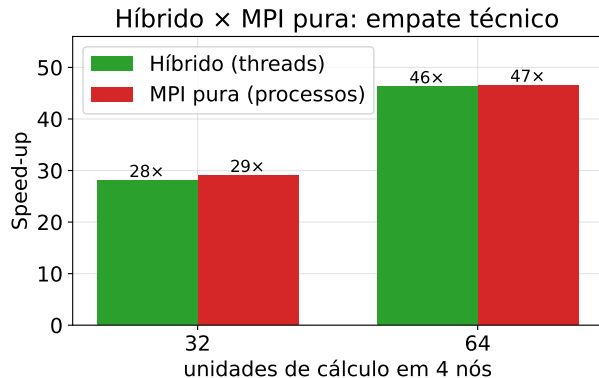


Coordenador **quase ocioso**, compartilha o nó 0 com um worker. *Onde alocá-lo?*

- **Dedicar 1 núcleo** (worker do nó 0 usa 15 threads): **2,09 s**.
- **Não dedicar** (16 threads + coordenador): 2,48 s — *oversubscrição* do nó.
- **Dedicar um nó** (n4): **4,04 s** — só 3 workers.

⇒ vale dedicar 1 **núcleo** (−16%), **nunca um nó** (+63%).

Híbrido × MPI pura



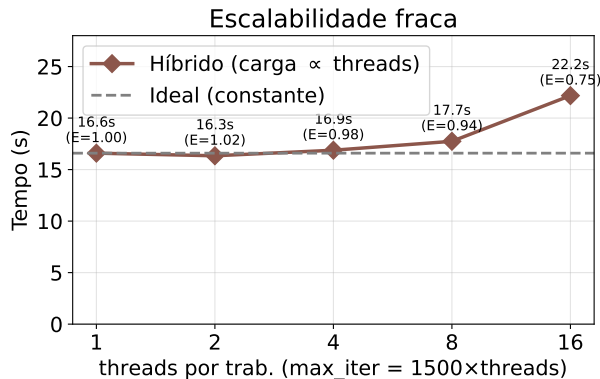
Mesma alocação (4 nós):

- 64 unidades: **2,01 s** (híbrido) vs **2,00 s** (MPI pura) — **empate técnico**.

Por quê? Problema *compute-bound* e independente: distribuir por *threads* ou por *processos* rende igual.

Vantagem do híbrido: 5 ranks MPI vs 64 $\Rightarrow$ menos memória e mensagens (cresce com a escala).

Escalabilidade fraca



Carga $\propto$ threads $\Rightarrow$ tempo deveria ser **constante**.

- Até 32 núcleos: 16,6 $\rightarrow$ 17,7 s, eficiência **0,94** — quase ideal.
- 64 núcleos: sobe a 22 s (0,75) — de novo o Hyper-Threading.

Confirma o bom balanceamento do **workpool** + distribuição MPI sob demanda.

- Cumprimos o modelo pedido: **MPI Coordenador/Trabalhador** (1 processo pesado/nó) + **workpool OpenMP sem mestre** (controle por estrutura de dados, 2 threads/núcleo com HT).
- **46×** de speed-up em 64 núcleos; escalabilidade **fraca quase ideal** até os núcleos físicos.
- A queda de eficiência é o **limite do Hyper-Threading**, não do algoritmo.
- Coordenador: vale dedicar **1 núcleo** (−16%, evita oversubscrição), **nunca um nó** inteiro (+63%).
- Híbrido **empata** com a MPI pura, usando **muito menos ranks** — vantagem em escala.

Saída validada **byte-idêntica** à sequencial. Código, relatório e dados no repositório.

Obrigado!

Perguntas?

Bernardo Zamin ■ João Pedro Cunha | Programação Paralela — PUCRS